

A PROJECT REPORT ON

TASKFLOW PRO

Enterprise Task Management System

A Full-Stack Role-Based Collaboration Platform for Team & Task Management

A Personal Project Report

Prepared And Developed

by

M.Kannan .MCA.,

COPYRIGHT

© 2025 Kannan M. All Rights Reserved.

This report and the accompanying software, **TaskFlow Pro – Enterprise Task Management System**, were independently conceived, designed, developed, and documented by **Kannan M** as a personal full-stack software development project.

No part of this report, its source code, design, documentation, screenshots, diagrams, or other materials may be reproduced, distributed, modified, published, or transmitted in any form without the prior written permission of the author, except for educational, research, or non-commercial purposes with appropriate acknowledgment.

The third-party technologies, frameworks, libraries, and tools used in this project—including Java, Spring Boot, React.js, MySQL, JWT, Maven, Git, GitHub, and other open-source technologies—remain the intellectual property of their respective owners.

About the Author

Kannan M is a Master of Computer Applications (MCA) graduate with a strong interest in enterprise software development, backend engineering, and full-stack web application development. He enjoys designing scalable, secure, and user-friendly applications that solve real-world business problems.

TaskFlow Pro was developed as a personal project to strengthen practical expertise in Java, Spring Boot, React.js, REST APIs, MySQL, JWT Authentication, and modern software engineering practices. Through this project, he gained hands-on experience in designing enterprise-level architectures, implementing role-based access control, developing secure RESTful APIs, integrating frontend and backend systems, and building responsive user interfaces.

He is passionate about continuous learning, writing clean and maintainable code, exploring new technologies, and developing software solutions that improve productivity and collaboration in organizations.

Author Information

Name : Kannan M

Qualification : Master of Computer Applications (MCA)

Role : Software Developer | Java Full-Stack Developer

Duration : Sep 2025 – Nov 2025

Email : knkannan70@gmail.com

GitHub : <https://github.com/knkannan70>

LinkedIn : <https://www.linkedin.com/in/kannan-m-0771562b2>

Portfolio : <https://portfolio-ten-gules-60.vercel.app>

TABLE OF CONTENTS

Chapter 1: Introduction	8
1.1 Background	8
1.2 Existing System	8
1.3 Problems in the Existing System	8
1.4 Proposed System	8
1.5 Objectives	8
1.6 Scope	9
1.7 Advantages	9
Chapter 2: Technology Stack	10
2.1 Backend Technologies	10
2.1.1 Java 17	10
2.1.2 Spring Boot 3 (3.3.5)	10
2.1.3 Spring Security + JWT (JSON Web Tokens)	10
2.1.4 Spring Data JPA (Hibernate)	10
2.1.5 Spring WebSocket + STOMP	10
2.1.6 Spring Mail	10
2.1.7 SpringDoc OpenAPI (Swagger)	10
2.1.8 MySQL 8.x	11
2.1.9 Apache Maven	11
2.2 Frontend Technologies	11
2.2.1 React 19 + Vite	11
2.2.2 React Router DOM (v7)	11
2.2.3 TanStack React Query (v5)	11
2.2.4 Material UI (MUI, v9) + Tailwind CSS	11
2.2.5 Recharts	11
2.2.6 STOMP.js + SockJS	11
2.2.7 Axios	11
2.2.8 React Hook Form	12
2.2.9 Framer Motion & React Hot Toast	12
2.3 Technology Stack Summary	12
Chapter 3: System Analysis	13
3.1 Functional Requirements	13
3.2 Non-Functional Requirements	13
3.3 User Roles	14
3.4 Use Cases	14

3.5 Modules	14
Chapter 4: System Design	15
4.1 Architecture Overview	15
4.2 Database Design	16
4.3 Entity–Relationship Overview.....	17
4.4 Module Diagram	18
4.5 Class Overview (Backend)	19
4.6 API Flow Explanation	21
Chapter 5: Implementation	22
5.1 Authentication & JWT Security	22
5.2 Registration & Employee ID Generation.....	22
5.3 Password Recovery (OTP-Based)	22
5.4 Team Management.....	22
5.5 Task Management.....	22
5.6 Comments, Mentions & Attachments	23
5.7 Task History (Audit Trail).....	23
5.8 Team Chat	23
5.9 Notification System.....	23
5.10 Dashboard & Analytics.....	23
5.11 Role-Based Access Control (RBAC).....	23
5.12 REST APIs & CSV Reports	24
5.13 Frontend Architecture	24
5.14 Backend Architecture Summary	24
Chapter 6: Screenshots	25
6.1 Authentication Screens.....	25
6.2 Manager Experience	26
6.3 Team Member Experience.....	31
Chapter 7: Testing	35
7.1 Unit Testing.....	35
7.2 Functional Testing.....	35
7.3 Sample Test Cases.....	35
7.4 Test Summary	36
Chapter 8: Results	37
8.1 Application Workflow	37
8.2 Achievements	37
8.3 Output.....	37
8.4 Benefits	37

8.5 Performance	37
Chapter 9: Future Enhancements	38
Chapter 10: Conclusion	39
References	40

ABSTRACT

Problem Statement

Modern organizations, whether small startups or growing enterprises, struggle to coordinate work across multiple teams using fragmented tools — spreadsheets for task tracking, generic chat apps for communication, and manual processes for team formation and reporting. This fragmentation leads to a lack of accountability, poor visibility into task progress, delayed communication, and an absence of a single, authoritative source of truth for who is doing what, by when, and with what priority. Existing generic project management tools are often either too simplistic to enforce organizational role hierarchies or too complex and expensive for small-to-mid-sized teams that need a focused, purpose-built solution.

Proposed Solution

TaskFlow Pro is a full-stack, production-ready Enterprise Task Management System that addresses these gaps through a strict two-role model — Manager and Team Member — enforced end-to-end via Role-Based Access Control (RBAC). The system allows Managers to create and own multiple teams, onboard members either by direct account creation or by Employee ID lookup, assign tasks with priority and deadline metadata, and track team-wide performance through a dedicated analytics dashboard. Team Members receive a personalized workspace scoped strictly to their own team and tasks, can update task status and progress, attach files, comment with @mention notifications, and collaborate through a persistent, real-time team chat. All state-changing operations are authenticated using stateless JSON Web Tokens (JWT), authorized using method-level Spring Security annotations, and pushed to connected clients in real time using WebSocket/STOMP messaging, ensuring that dashboards, notifications, and chat views stay synchronized without manual refreshes.

Technologies Used

The backend is implemented in Java 17 using Spring Boot 3, with Spring Security and JWT for stateless authentication, Spring Data JPA (Hibernate) for persistence against a MySQL relational database, and Spring WebSocket for STOMP-based real-time messaging. Spring Mail integrates OTP-based password recovery and email notifications. The frontend is a single-page application built with React 19 and Vite, using React Router for client-side routing, TanStack React Query for server-state caching, Material UI (MUI) components layered over a custom Tailwind CSS design system, Recharts for dashboard visualizations, Axios with an automatic token-refresh interceptor, and STOMP.js with SockJS for the real-time client connection. The API is self-documented using SpringDoc OpenAPI (Swagger UI), and the build is managed with Apache Maven on the backend and npm/Vite on the frontend.

Expected Benefits

- Clear accountability through strict ownership validation — Managers can only act on teams and tasks they own, and Members are scoped strictly to their own team.
- Real-time visibility into task status, team performance, and notifications without manual page refreshes.
- Reduced administrative overhead through auto-generated Employee IDs, Team IDs, bulk task operations, and one-click CSV reporting.
- A complete, tamper-evident audit trail of every task field change, supporting transparency and post-hoc review.
- A secure-by-default architecture using industry-standard practices — BCrypt password hashing, short-lived access tokens, refresh-token rotation, and OTP-based password resets.

- A modular, layered codebase (Controller → Service → Repository) that is straightforward to extend with the future enhancements identified in Chapter 9.

Chapter 1: Introduction

1.1 Background

Task and team management is a foundational requirement for any organization that coordinates work across more than one person. As teams grow, informal coordination through email threads, spreadsheets, and instant-messaging groups becomes unreliable: tasks are forgotten, priorities are unclear, and there is no single record of who changed what and when. TaskFlow Pro was conceived to replace this ad-hoc coordination with a structured, role-aware digital workspace where every team, task, comment, and status change is recorded, attributable, and visible to the right people in real time.

1.2 Existing System

Prior to adopting a dedicated system, most small and mid-sized teams rely on a combination of the following:

- Spreadsheets (e.g., Excel/Google Sheets) for tracking task lists, manually updated by whoever remembers to do so.
- General-purpose chat applications (WhatsApp, Slack free tier) for team communication, disconnected from the actual task data.
- Email for status updates, approvals, and reminders, with no structured audit trail.
- Manual, verbal, or ad-hoc processes for onboarding new team members and assigning them to a reporting structure.

1.3 Problems in the Existing System

- No single source of truth: task status, ownership, and history are scattered across tools.
- No enforced access control: anyone with access to a shared file can alter any record, regardless of role.
- No real-time visibility: managers must manually poll team members for updates instead of seeing live dashboards.
- No audit trail: it is impossible to reconstruct who changed a task's priority, due date, or status, and when.
- Poor scalability: spreadsheet-based tracking breaks down as the number of teams, members, and tasks grows.
- Fragmented communication: task-specific discussion happens outside the context of the task itself, making history hard to follow.

1.4 Proposed System

TaskFlow Pro is proposed as a unified, web-based platform that consolidates team formation, task assignment, real-time collaboration, notification delivery, and analytics reporting into a single, secure application. The system is built around two clearly defined roles — Manager and Team Member — with every backend endpoint protected by method-level authorization (@PreAuthorize) and every frontend route protected by client-side route guards, so that access control is enforced consistently at both layers.

1.5 Objectives

1. Design and implement a secure, JWT-based authentication and authorization system with distinct Manager and Team Member roles.
2. Enable Managers to create, update, and manage multiple teams with full ownership validation on every mutation.
3. Provide a complete task lifecycle — creation, assignment, status update, commenting, file attachment, and audit history — scoped correctly to team membership.

4. Deliver real-time updates for dashboards, notifications, and team chat using WebSocket/STOMP messaging.
5. Provide role-specific analytics dashboards with visual charts and downloadable CSV reports.
6. Ensure the system is production-ready: transactional integrity, optimistic locking, cascading deletes, and CORS-restricted APIs.

1.6 Scope

The scope of the system covers the complete task-and-team management workflow for a single organization instance: user registration and authentication, team creation and membership management, task creation and lifecycle management (including comments, attachments, and history), a real-time team chat per team, an in-app notification system, role-specific dashboards, and CSV export of task and team data. The system is designed for a single-tenant deployment (one organization per running instance) and does not currently include multi-tenant organization isolation, mobile native applications, or third-party project-management tool integrations — several of these are identified as future enhancements in Chapter 9.

1.7 Advantages

- Strict two-role RBAC keeps administrative power and day-to-day task execution cleanly separated.
- Real-time architecture (WebSocket/STOMP) keeps dashboards, notifications, and chat synchronized without manual refreshing.
- Comprehensive audit history on every task field change supports accountability and traceability.
- Modular, layered backend architecture (Controller → Service → Repository → Entity) makes the codebase easy to maintain and extend.
- Self-documenting REST API via Swagger UI simplifies onboarding for future developers.
- Secure-by-default practices: BCrypt password hashing, short-lived JWT access tokens, refresh-token rotation, and OTP-based password recovery.

Chapter 2: Technology Stack

TaskFlow Pro follows a clear separation between a Java-based backend REST API and a React-based single-page frontend application, communicating over HTTP (REST) and WebSocket (STOMP). This chapter documents every major technology actually present in the source code and the engineering rationale for choosing it.

2.1 Backend Technologies

2.1.1 Java 17

Java 17 is the Long-Term Support (LTS) version specified in the project's Maven configuration. It provides a mature, statically typed, and highly performant runtime with strong tooling support, making it a dependable choice for a transactional, business-logic-heavy backend such as this one.

2.1.2 Spring Boot 3 (3.3.5)

Spring Boot 3 was chosen to minimize boilerplate configuration while providing an opinionated, production-ready foundation. It auto-configures the embedded Tomcat server, dependency injection container, and integrates cleanly with Spring Security, Spring Data JPA, and Spring WebSocket — all of which are used extensively in this project.

2.1.3 Spring Security + JWT (JSON Web Tokens)

Spring Security provides the authentication/authorization filter chain, while the JWT library issues and validates stateless JSON Web Tokens. This combination allows the API to remain stateless (no server-side session storage) while still enforcing fine-grained, role-based method security via `@PreAuthorize` annotations on every controller endpoint.

2.1.4 Spring Data JPA (Hibernate)

Spring Data JPA, backed by Hibernate as the ORM provider, was chosen to reduce hand-written SQL and to model the 16-entity relational schema (users, teams, tasks, and their associated child entities) using declarative annotations, repository interfaces, and JPQL/`@Query` methods, including `@EntityGraph` for eager, N+1-free fetching in performance-sensitive queries such as team chat retrieval.

2.1.5 Spring WebSocket + STOMP

Spring's WebSocket support with the STOMP sub-protocol was selected to enable real-time, bidirectional communication between the server and connected clients. This powers automatic dashboard refreshes, live notification delivery, and real-time redirect behavior when a member is removed from a team.

2.1.6 Spring Mail

Spring Mail, configured against an SMTP/Gmail provider, is used to deliver transactional emails: welcome messages on registration, task-assignment notifications, and OTP codes for password recovery.

2.1.7 SpringDoc OpenAPI (Swagger)

SpringDoc OpenAPI auto-generates an interactive Swagger UI directly from the controller annotations, giving developers and testers a live, browsable reference for every REST endpoint without maintaining separate documentation.

2.1.8 MySQL 8.x

MySQL was selected as the relational database of choice for its maturity, wide hosting support, and strong fit for the strictly relational, foreign-key-driven schema (users, teams, tasks, and their many child tables) that this system requires.

2.1.9 Apache Maven

Maven manages backend dependencies, build lifecycle, and packaging, providing a standard, declarative build configuration (pom.xml) that is easy for any Java developer to pick up.

2.2 Frontend Technologies

2.2.1 React 19 + Vite

React 19 is used to build the entire single-page application as a component tree, while Vite serves as the build tool and development server, offering near-instant hot-module-reload during development and an optimized production bundle.

2.2.2 React Router DOM (v7)

React Router handles all client-side navigation, including the `ProtectedRoute` and `PublicRoute` wrapper components that gate access to authenticated-only or guest-only pages based on the current session state.

2.2.3 TanStack React Query (v5)

React Query manages server-state: caching API responses, automatically refetching stale data, and de-duplicating concurrent requests, which significantly reduces the amount of manual state-management code required for data fetched from the backend.

2.2.4 Material UI (MUI, v9) + Tailwind CSS

The interface layers Material UI's accessible, pre-built component primitives (dialogs, menus, form controls) with a custom Tailwind CSS utility-class design system and a dedicated CSS-variables theme (dark background, red accent palette) defined in the application's global stylesheet, giving the product a distinctive, consistent visual identity beyond MUI's defaults.

2.2.5 Recharts

Recharts renders the dashboard's pie chart (priority distribution) and bar chart (monthly completion trend) using a declarative, React-idiomatic charting API.

2.2.6 STOMP.js + SockJS

On the client side, STOMP.js together with SockJS establishes and maintains the WebSocket connection to the backend's `/ws` endpoint, subscribing to topic and queue destinations for live dashboard, notification, and chat updates.

2.2.7 Axios

Axios is the HTTP client for all REST calls, configured with a request/response interceptor that automatically attaches the JWT access token to outgoing requests and silently refreshes an expired token using the stored refresh token.

2.2.8 React Hook Form

React Hook Form manages form state and validation across the application's numerous forms (login, registration, task creation, profile editing) with minimal re-renders and a straightforward validation API.

2.2.9 Framer Motion & React Hot Toast

Framer Motion provides smooth page and component transition animations, while React Hot Toast surfaces lightweight, non-blocking success/error notifications throughout the user interface.

2.3 Technology Stack Summary

Layer	Technology	Version
Frontend Framework	React	19.x
Frontend Build Tool	Vite	8.x
Frontend Routing	React Router DOM	v7
Server State	TanStack React Query	v5
UI Components	Material UI (MUI)	v9
Styling	Tailwind CSS + CSS Variables	v3
Animation	Framer Motion	v12
Charts	Recharts	v3
Real-Time Client	STOMP.js + SockJS	Latest
HTTP Client	Axios	v1.18
Notifications (UI)	React Hot Toast	v2.6
Backend Framework	Spring Boot	3.3.5
Language	Java	17 (LTS)
Security	Spring Security + JWT	—
ORM	Spring Data JPA (Hibernate)	—
Real-Time Server	Spring WebSocket + STOMP	—
Email	Spring Mail (SMTP/Gmail)	—
API Documentation	SpringDoc OpenAPI (Swagger)	Latest
Database	MySQL	8.x
Build Tool	Apache Maven	3.x

Chapter 3: System Analysis

3.1 Functional Requirements

- The system shall allow a new user to register as either a Manager or a Team Member, receiving an auto-generated Employee ID (e.g., MGR0001 or EMP0002).
- The system shall authenticate users via email and password, returning a short-lived access token (15 minutes) and a long-lived refresh token (30 days).
- The system shall allow a Manager to create, update, and delete teams that they own, with cascading cleanup of members, chat history, and task assignments on deletion.
- The system shall allow a Manager to add members to a team either by creating a new account directly or by looking up an existing employee by Employee ID.
- The system shall allow a Team Member to self-join a single active team using a Team ID, and to leave that team voluntarily.
- The system shall allow a Manager to create tasks (with title, description, priority, start/due dates, and estimated hours) that are automatically assigned to all active members of the target team.
- The system shall allow both roles to update task status, with Team Members restricted from setting the Cancelled status.
- The system shall support commenting on tasks with @mention support that triggers a targeted notification, and file attachments up to 5 MB per file.
- The system shall maintain a complete audit history for every task field change, recording who changed it, when, and the old/new values.
- The system shall provide a persistent, real-time team chat per team, supporting send, edit, delete, and pin operations.
- The system shall push real-time notifications for task and team lifecycle events via WebSocket, and maintain an in-app notification center with read/unread state.
- The system shall provide role-specific analytics dashboards and CSV export of task and team data (Manager only).

3.2 Non-Functional Requirements

- Security: All state-changing endpoints must be authenticated via JWT and authorized via role-based method security; passwords must be hashed using BCrypt.
- Performance: Frequently accessed relational queries (e.g., team chat) must use @EntityGraph to avoid N+1 query patterns.
- Consistency: Concurrent task edits must be protected using optimistic locking (@Version) to prevent lost updates.
- Reliability: Multi-step mutations (e.g., team deletion with cascading cleanup) must be wrapped in @Transactional boundaries to guarantee atomicity.
- Usability: The interface must clearly differentiate Manager and Team Member views and gracefully handle empty states (e.g., a member with no team).
- Portability: The backend must run on any environment with Java 17 and MySQL 8 installed; the frontend must build to static assets deployable behind any web server.

3.3 User Roles

Role	Description
Manager	Creates and owns one or more teams, manages membership, creates and assigns tasks, monitors team-wide analytics, downloads reports, and moderates team chat.
Team Member	Belongs to at most one team at a time, works on assigned tasks, updates personal task status/progress, comments and attaches files, and participates in team chat.

3.4 Use Cases

- A Manager registers, creates a team, and adds members either by creating new accounts or by Employee ID lookup.
- A Manager creates a task for a team; the system auto-assigns it to every active team member and emails each of them.
- A Team Member logs in, views their personalized dashboard, and updates the status of an assigned task as they progress through it.
- A Team Member comments on a task and @mentions a colleague, who receives a targeted notification.
- A Manager reviews the analytics dashboard, identifies overdue tasks, and downloads a CSV report for a stakeholder review.
- Team members and their Manager communicate in the team's chat channel regarding an in-progress task.

3.5 Modules

Module	Primary Responsibility
Authentication	Registration, login, token refresh, logout, OTP-based password reset.
Team Management	Team CRUD, membership (add/remove/join/leave), ownership validation.
Task Management	Task CRUD, status/progress updates, duplication, bulk actions, archiving.
Comments & Attachments	Threaded task comments with mentions; file uploads up to 5 MB.
Task History	Immutable audit log of every task field change.
Team Chat	Real-time messaging per team with edit/delete/pin support.
Notifications	Event-driven, real-time in-app notification center.
Dashboard & Reports	Role-specific analytics and CSV export.
Profile & Settings	Profile editing, avatar/file upload, password change.

Chapter 4: System Design

4.1 Architecture Overview

TaskFlow Pro follows a classic layered client–server architecture. The React single-page application communicates with a Spring Boot REST API over HTTPS for standard CRUD operations, and over a WebSocket/STOMP channel for real-time push events. The backend itself is organized into four internal layers — Security, Controller, Service, and Repository — each with a single, well-defined responsibility, sitting above a MySQL relational database accessed through JDBC via Hibernate.

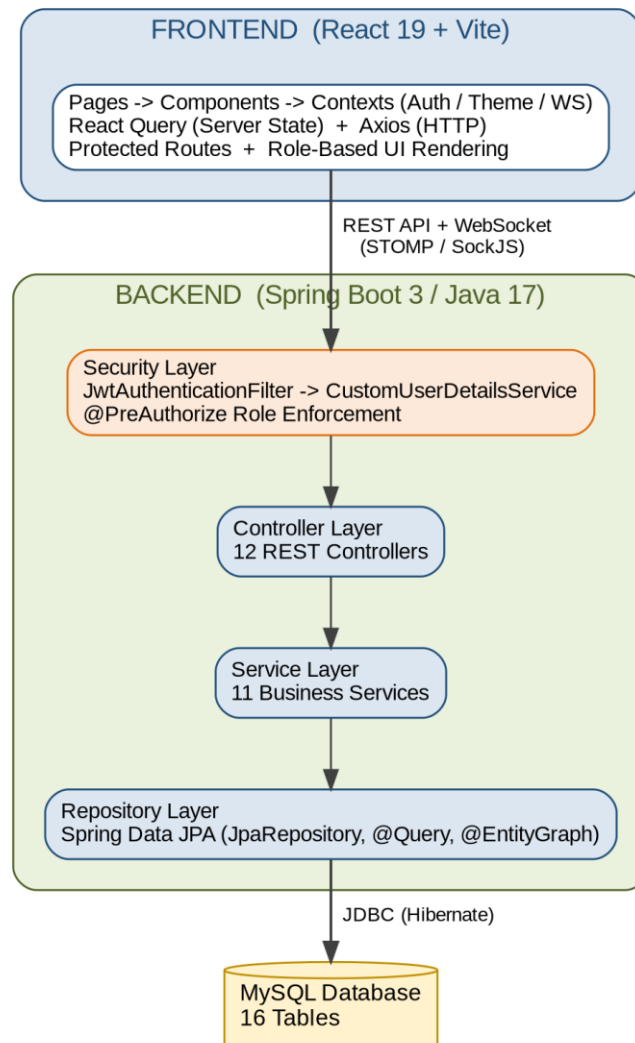


Figure 4.1 – Layered Client–Server Architecture

4.2 Database Design

The relational schema is composed of 16 tables that model users, organizational teams, the task lifecycle, and their supporting audit/communication data. Referential integrity is enforced through foreign keys with cascading deletes where appropriate (for example, deleting a team cascades to its memberships, chat history, and task assignments).

Table	Description	Key Relationships
users	All employees (Managers & Members); profile, role, and auth data.	→ teams (leader), team_members, tasks, chat_messages
teams	Organizational teams with name, description, color, leader.	→ team_members, tasks, chat_messages
team_members	Active membership of a user in a team.	↔ teams + users
tasks	Core tasks with status, priority, dates, progress.	→ task_assignments, comments, attachments, history
task_assignments	Maps users to the tasks they are assigned to.	↔ tasks + users
task_comments	User comments on tasks with mention support.	↔ tasks + users
task_attachments	Files attached to tasks (path, size, type).	↔ tasks + users
task_history	Full audit log of every task field change.	↔ tasks + users
task_tags	Many-to-many join between tasks and tags.	↔ tasks + tags
chat_messages	Team chat messages with edit, delete, pin support.	↔ teams + users
notifications	In-app event notifications per user.	↔ users
categories	Task categories for organizational grouping.	→ tasks
tags	Reusable labels for tasks.	↔ tasks via task_tags
refresh_tokens	Stored refresh tokens for session management.	↔ users
password_reset_tokens	Time-limited OTP tokens for password reset.	↔ users
user_settings	Per-user application preferences.	↔ users

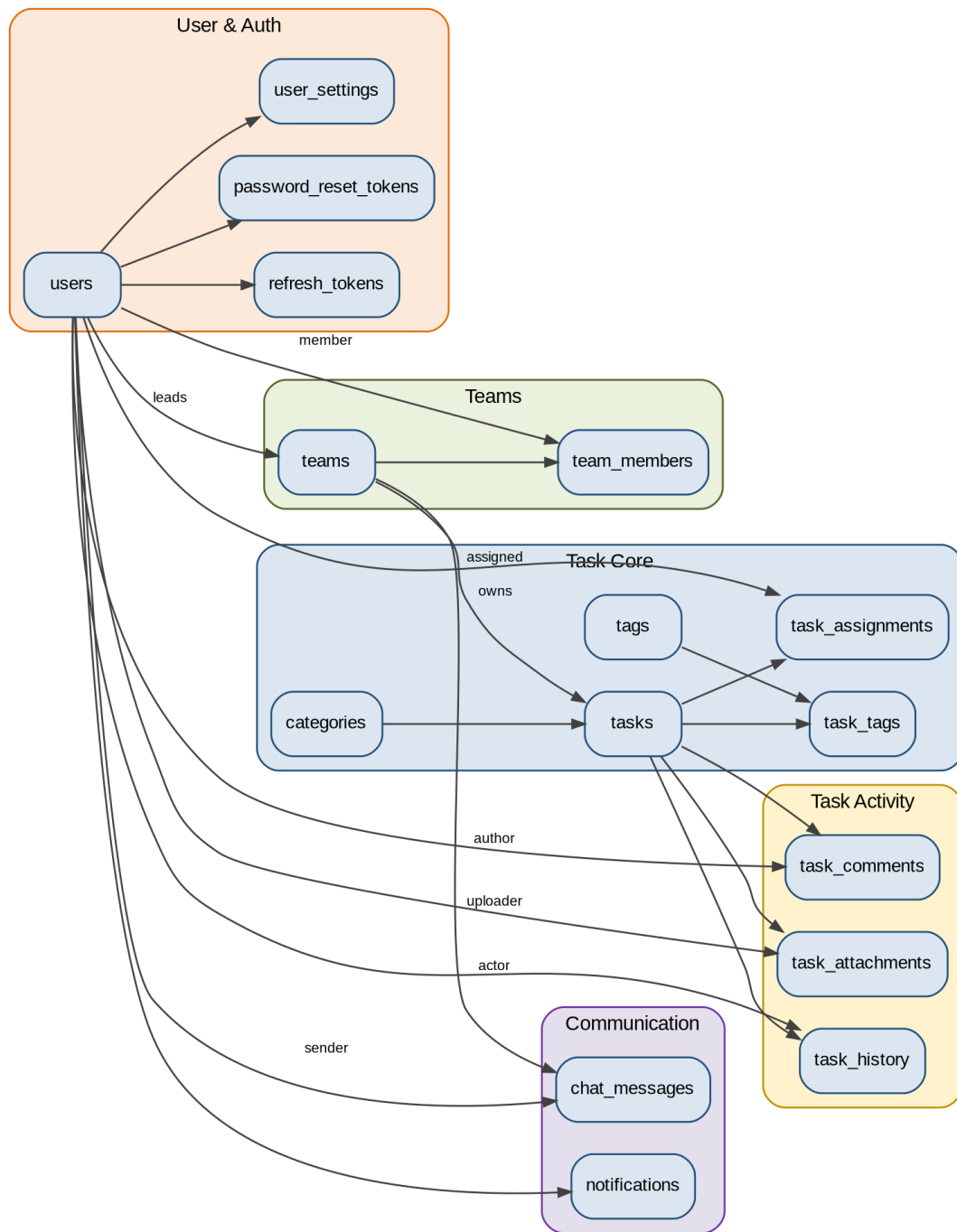


Figure 4.2 – Database Schema Grouped by Functional Area

4.3 Entity–Relationship Overview

At the core of the schema, a User has a single Role (ROLE_MANAGER or ROLE_TEAM_MEMBER) and, through the team_members join table, an active membership in at most one Team. Each Team has exactly one leader (a Manager). A Task belongs to exactly one Team and is linked, through task_assignments, to every Team Member who is expected to act on it. Comments, attachments, and history entries all reference both the parent Task and the acting User, ensuring every piece of activity is fully attributable.

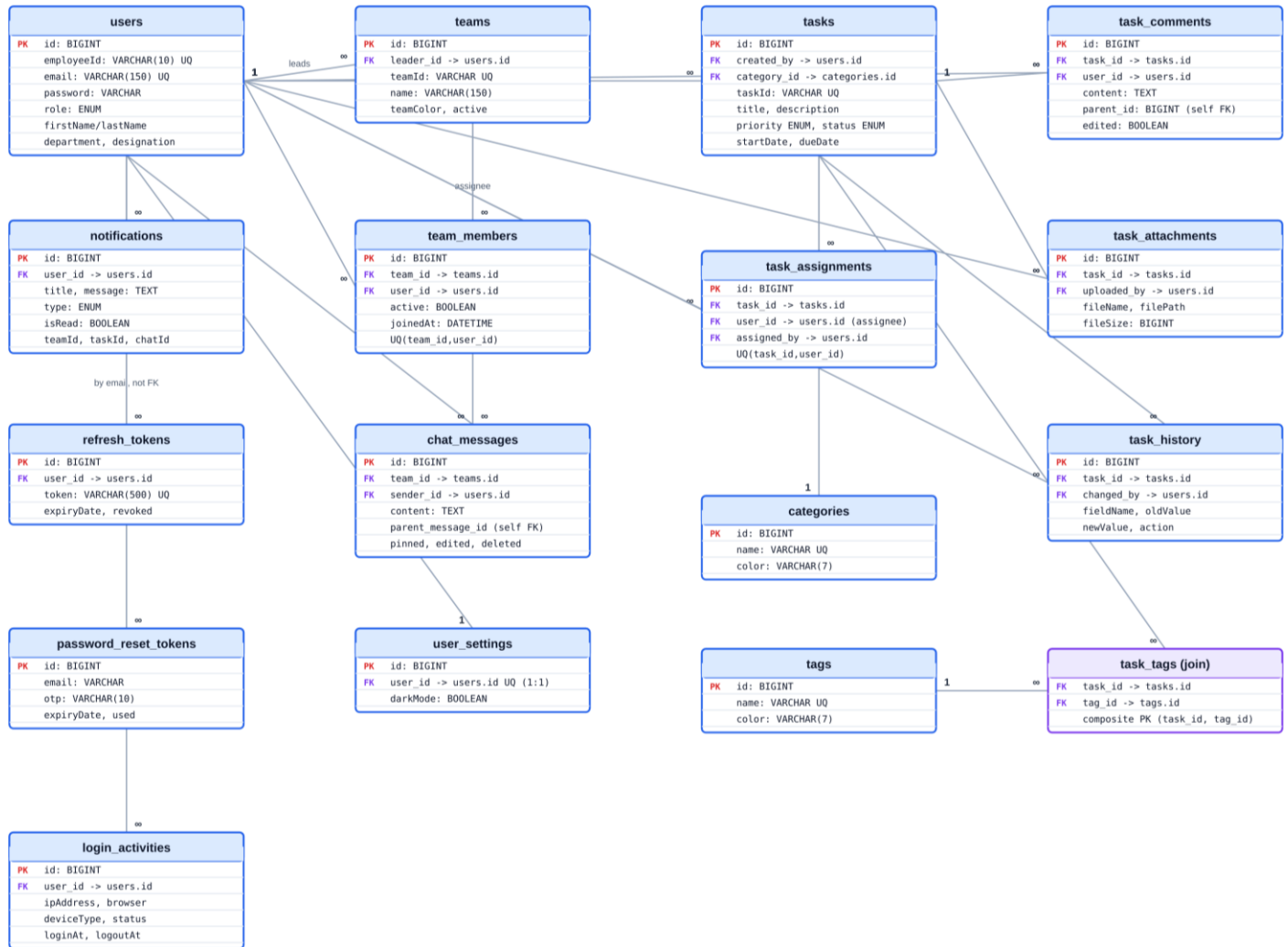


Figure 4.3 — Entity relationship diagram (crow's-foot notation)

4.4 Module Diagram

The system is organized into functional modules that build on the Auth module for identity and authorization, and compose together to deliver task management, team collaboration, and real-time communication:

- Auth Module – issues JWT, manages refresh & OTP reset
- Team Module – team CRUD, membership, ownership checks
- Task Module – task CRUD, status/progress, bulk ops
- Comment/Attachment – nested under Task Module
- History Module – write-only audit trail, read by both roles
- Chat Module – per-team messaging, STOMP broadcast
- Notification Module – event bus consumer, STOMP push, REST read
- Dashboard Module – aggregated read-only analytics per role
- Profile/Settings – self-service account management

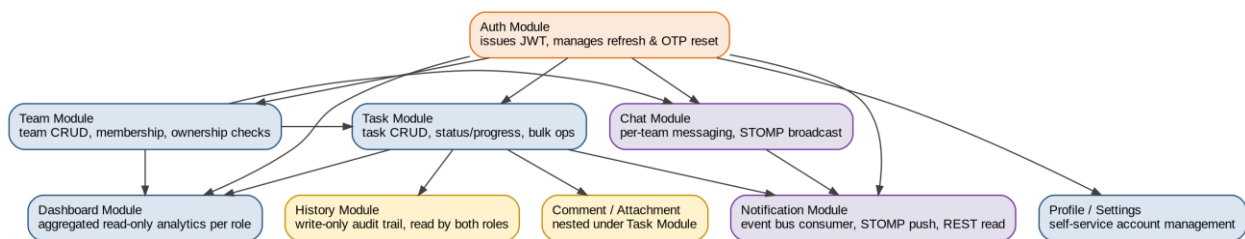


Figure 4.4 – Module Dependency Diagram

4.5 Class Overview (Backend)

The backend follows a strict separation of concerns. Controllers are thin and delegate all business logic to Services; Services orchestrate one or more Repositories and enforce ownership/authorization rules that go beyond simple role checks (for example, verifying a Manager owns the specific team being mutated); Repositories are Spring Data JPA interfaces that either derive queries from method names or use `@Query/@EntityGraph` for more complex or performance-sensitive access patterns.

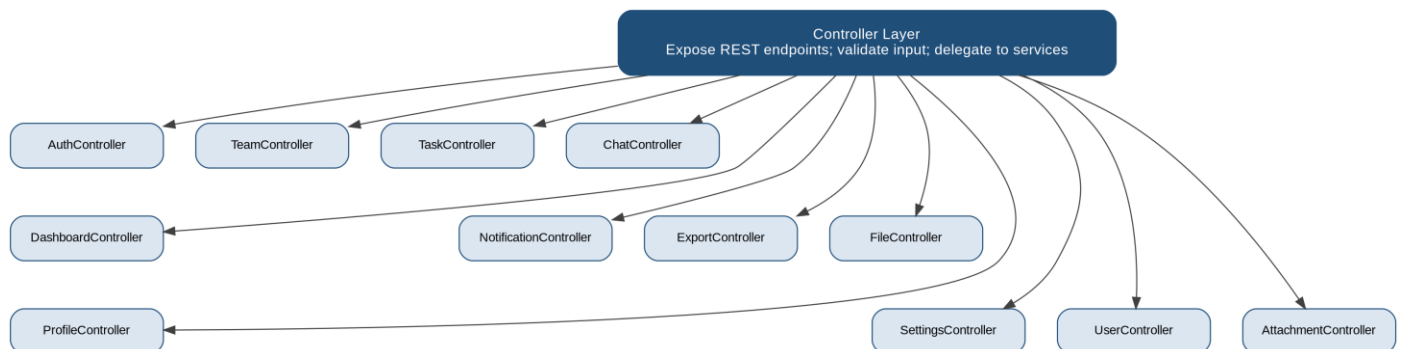


Figure 4.5.1 – Controller Layer Class Overview

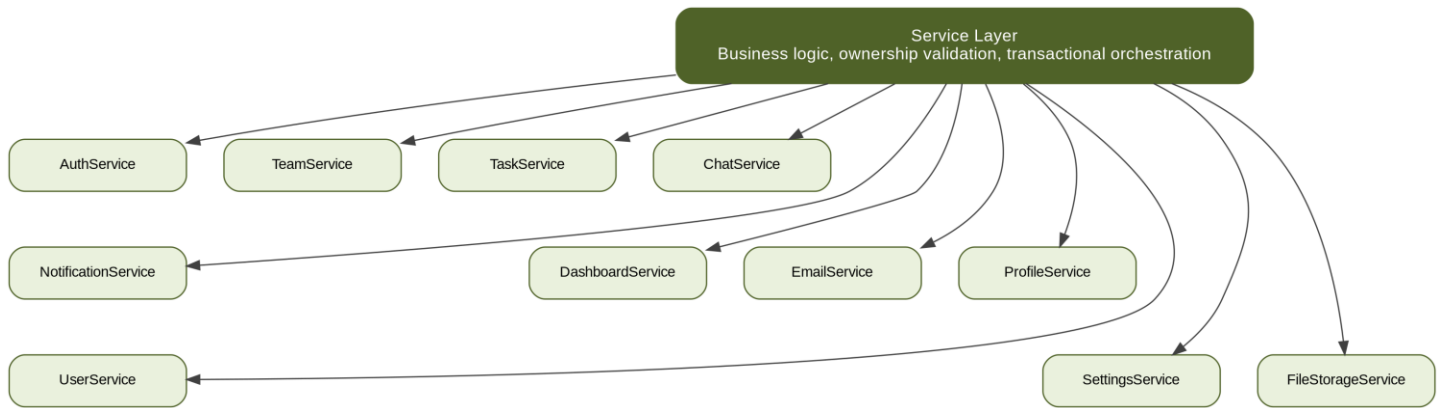


Figure 4.5.2 – Service Layer Class Overview

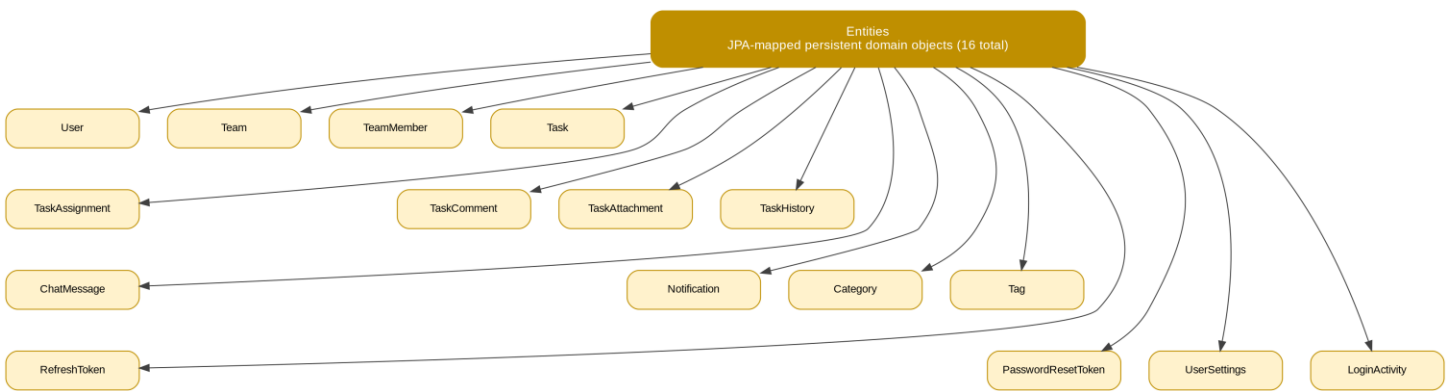


Figure 4.5.3 – Entity (JPA) Class Overview

Class Group	Representative Classes	Responsibility
Controllers	AuthController, TeamController, TaskController, ChatController, DashboardController, NotificationController, ExportController, FileController, ProfileController, SettingsController, UserController, AttachmentController	Expose REST endpoints; validate input; delegate to services
Services	AuthService, TeamService, TaskService, ChatService, NotificationService, DashboardService, EmailService, ProfileService, UserService, SettingsService, FileStorageService	Business logic, ownership validation, transactional orchestration
Entities	User, Team, TeamMember, Task, TaskAssignment, TaskComment, TaskAttachment, TaskHistory, ChatMessage, Notification, Category, Tag, RefreshToken, PasswordResetToken, UserSettings, LoginActivity	JPA-mapped persistent domain objects (16 total)
Security	JwtTokenProvider, JwtAuthenticationFilter, CustomUserDetailsService, AuthEntryPoint	Token issuance/validation, request filtering, unauthorized-access handling

4.6 API Flow Explanation

A typical authenticated request flows as follows: the React frontend attaches the JWT access token to the Authorization header via an Axios interceptor; the request reaches `JwtAuthenticationFilter`, which validates the token's signature and expiry and loads the user's authorities via `CustomUserDetailsService`; Spring Security's `@PreAuthorize` annotations on the target controller method then verify the caller's role is permitted to invoke that operation; the controller delegates to the corresponding service, which performs any additional ownership checks (e.g., "does this Manager own this team?") before calling the repository layer; the result is serialized into a DTO and returned as a uniform `ApiResponse` or `PagedResponse` envelope. For real-time flows, services additionally publish an event over the STOMP message broker (/topic or /queue destinations), which the frontend's `WebSocketContext` consumes to update dashboards, notifications, or chat views without a page refresh.

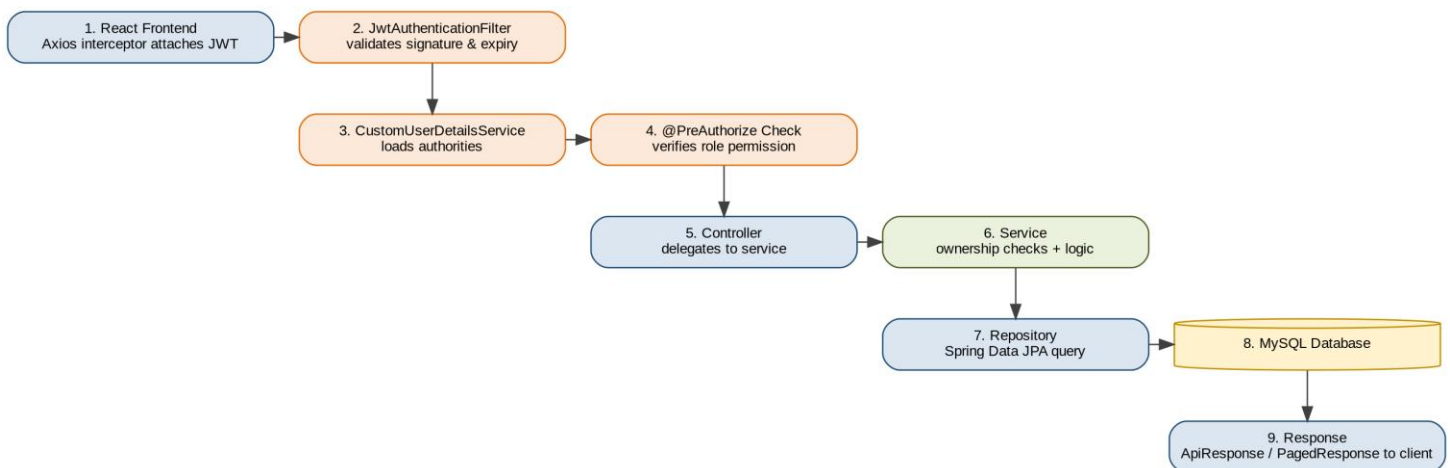


Figure 4.6.2 – Authenticated REST API Request Flow

For real-time flows, services additionally publish an event over the STOMP message broker (/topic or /queue destinations), which the frontend's `WebSocketContext` consumes to update dashboards, notifications, or chat views without a page refresh.

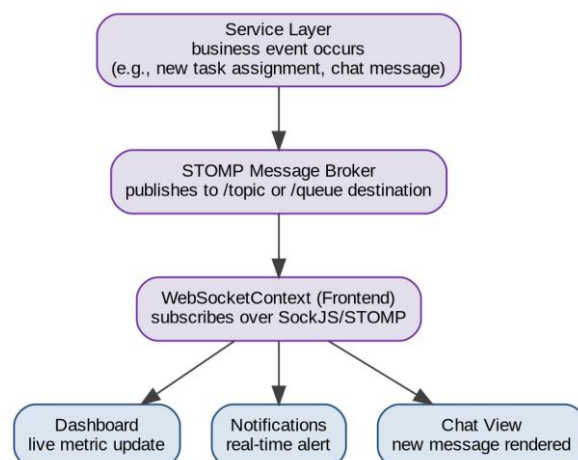


Figure 4.6.2 – Real-Time WebSocket/STOMP Push Flow

Chapter 5: Implementation

This chapter documents how each major module is actually implemented in the source code, covering only functionality verified against the backend and frontend repositories.

5.1 Authentication & JWT Security

AuthController exposes `/auth/login`, `/auth/register`, `/auth/refresh-token`, `/auth/forgot-password`, `/auth/reset-password`, and `/auth/logout`. On login, AuthService verifies the submitted credentials, and on success issues a short-lived access token (15 minutes) and a 30-day refresh token, both generated and validated by JwtTokenProvider. Every incoming request passes through JwtAuthenticationFilter, which extracts and validates the bearer token before Spring Security's authorization checks run. On the frontend, an Axios response interceptor automatically detects a 401 response caused by an expired access token, silently calls the refresh endpoint, and retries the original request — keeping the user session alive without any visible interruption.

5.2 Registration & Employee ID Generation

On registration, AuthService assigns the requested role (`ROLE_MANAGER` or `ROLE_TEAM_MEMBER`) and generates a unique, sequential Employee ID with a role-specific prefix — MGR for Managers and EMP for Team Members (e.g., `MGR0001`, `EMP0002`) — which is then used throughout the system as the canonical human-readable identifier for lookups such as “add member by Employee ID.” A welcome notification and email are dispatched immediately after successful registration.

5.3 Password Recovery (OTP-Based)

The forgot-password flow generates a 6-digit one-time password, persists it in the `password_reset_tokens` table with a 10-minute expiry, and emails it to the user via EmailService. The reset-password endpoint validates the submitted OTP against this record before allowing the password to be updated, after which the token is invalidated.

5.4 Team Management

TeamService enforces that every mutating operation — update, delete, add member, remove member — first verifies that the requesting Manager is the owner (leader) of the target team, rejecting any cross-team mutation attempt. Team creation auto-generates a unique Team ID (e.g., `TEAM-0001`). Deleting a team is a cascading, transactional operation that removes all `team_members` rows, `chat_messages`, and `task_assignments` associated with that team, and broadcasts a `TEAM_DELETED` notification to every affected member in real time.

5.5 Task Management

TaskService implements the full task lifecycle: creation (with automatic assignment to every active member of the target team and an email to each), status transitions (`PENDING` → `IN_PROGRESS` → `COMPLETED`, with Managers additionally able to set `CANCELLED`), progress auto-updates (0% on Pending, 100% on Completed), duplication (copies a task as a new Pending task with a 7-day default due date), bulk status updates and bulk deletion, and archive/restore for decluttering completed work without permanent loss. Every task carries an `@Version` field, which Hibernate uses for optimistic locking to prevent two concurrent edits from silently overwriting one another.

5.6 Comments, Mentions & Attachments

Team members and managers can comment on any task they have access to; comments support @employeeid mention syntax, which TaskService parses to dispatch a targeted MENTION notification to the mentioned user in addition to the general TASK_COMMENT_ADDED notification sent to the rest of the team. FileStorageService handles attachment uploads (images, PDFs, Office documents, and ZIP archives) up to 5 MB per file, storing metadata (path, size, content type) in the task_attachments table.

5.7 Task History (Audit Trail)

Every field-level change to a task — status, priority, due date, assignee changes, and more — is written to the task_history table by TaskService as part of the same transaction as the change itself, recording the acting user, a timestamp, and the old and new values. Both roles can view this history, giving Team Members read-only visibility and Managers full traceability for accountability purposes.

5.8 Team Chat

ChatService persists every message to chat_messages and uses @EntityGraph on the repository to eagerly fetch sender details in a single query, eliminating N+1 query overhead when loading a chat thread. Authors can edit or delete their own messages; Managers can delete any message within their owned teams, or wipe an entire team's chat history. Every new message triggers a CHAT_MESSAGE notification to the rest of the team and is broadcast over the WebSocket/STOMP channel so it appears instantly for connected clients.

5.9 Notification System

NotificationService centralizes the creation of all in-app notifications — covering task lifecycle events, team lifecycle events, mentions, chat messages, and account events (welcome, password reset) — and pushes each one over STOMP in addition to persisting it for later retrieval. The frontend maintains an unread-count badge, supports marking individual or all notifications as read, and supports deleting individual or all notifications.

5.10 Dashboard & Analytics

DashboardController exposes distinct /dashboard/leader and /dashboard/member endpoints. DashboardService aggregates task counts by status and priority, computes monthly completion trends for the current year, and — for Managers — produces a per-team performance breakdown (task counts, completion rate, member count). The frontend renders this data using Recharts (pie chart for priority distribution, bar chart for monthly completions) and refreshes automatically on relevant WebSocket events.

5.11 Role-Based Access Control (RBAC)

Authorization is enforced at two levels: on the backend, every controller method carries an @PreAuthorize("hasRole('MANAGER')") or equivalent annotation restricting it to the appropriate role, in addition to service-level ownership checks; on the frontend, ProtectedRoute and role-aware conditional rendering prevent Team Members from ever seeing Manager-only UI (team creation, bulk task actions, CSV export buttons), even though the definitive enforcement always happens server-side.

5.12 REST APIs & CSV Reports

The system exposes a consistent, resource-oriented REST API across 12 controllers, documented automatically via SpringDoc OpenAPI (Swagger UI) at `/api/swagger-ui.html`. ExportController generates on-demand CSV reports — a Tasks report (Task ID, Title, Status, Priority, Due Date) and a Teams report (Team ID, Name, Leader Email) — restricted to Managers via `@PreAuthorize`, streamed directly as a downloadable text/csv response without any intermediate file storage.

5.13 Frontend Architecture

The React application is organized around three context providers — AuthContext (session/token state), ThemeContext (light/dark theme), and WebSocketContext (STOMP connection lifecycle and subscriptions) — which wrap a page tree routed by React Router. Server data fetching and caching is centralized through TanStack React Query hooks, and all HTTP calls flow through a single configured Axios instance (`services/api.js`) that transparently attaches and refreshes JWT tokens.

5.14 Backend Architecture Summary

The backend cleanly separates concerns across config (security, Swagger, WebSocket configuration), controller, dto, entity, enums, exception, repository, security, and service packages — 12 REST controllers, 16 JPA entities, and 11 business services in total — which keeps each class focused on a single responsibility and makes the codebase straightforward to navigate and extend.

Chapter 6: Screenshots

This chapter presents representative screenshots captured from a running instance of TaskFlow Pro, covering both the Manager and Team Member experiences across authentication, task management, team collaboration, and reporting. Each figure is captioned and briefly explained below.

6.1 Authentication Screens

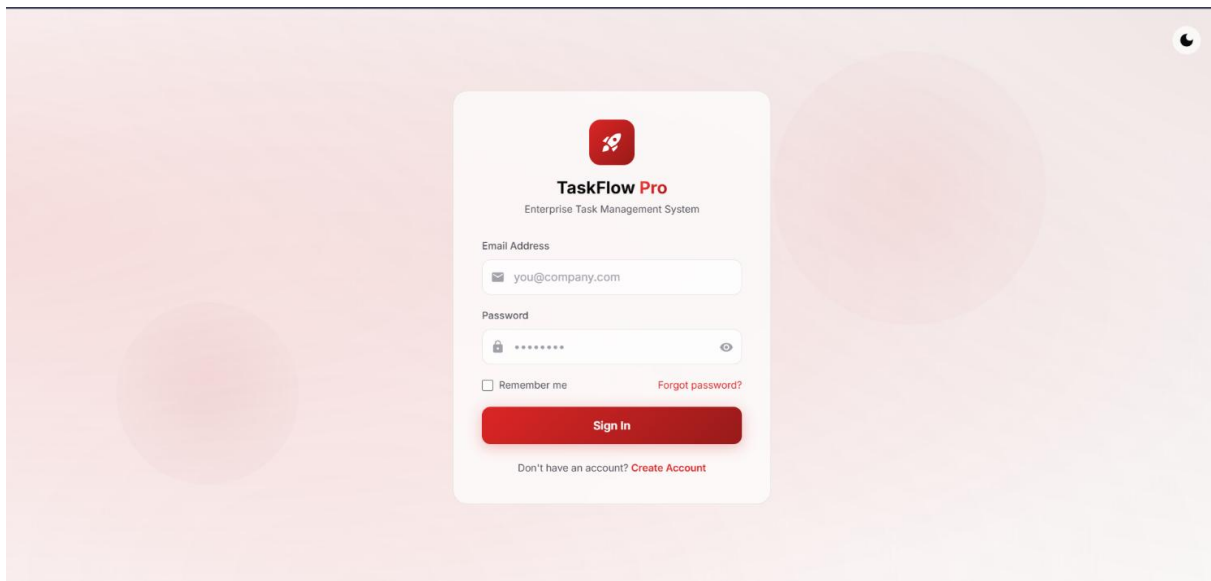
A screenshot of the TaskFlow Pro login page. The page has a light pink background with a subtle pattern of overlapping circles. In the center is a white rounded rectangle containing the login form. At the top of the form is the TaskFlow Pro logo (a red square with a white rocket icon) and the text "TaskFlow Pro" and "Enterprise Task Management System". Below this are two input fields: "Email Address" with the placeholder "you@company.com" and "Password" with a masked password "*****" and an eye icon to toggle visibility. There is a "Remember me" checkbox and a "Forgot password?" link. A red "Sign In" button is at the bottom of the form. Below the button is a link: "Don't have an account? Create Account".

Figure 6.1: Login Page

The authentication entry point of TaskFlow Pro, presenting a dark-themed interface where registered users sign in with their email and password to receive a JWT access and refresh token pair.

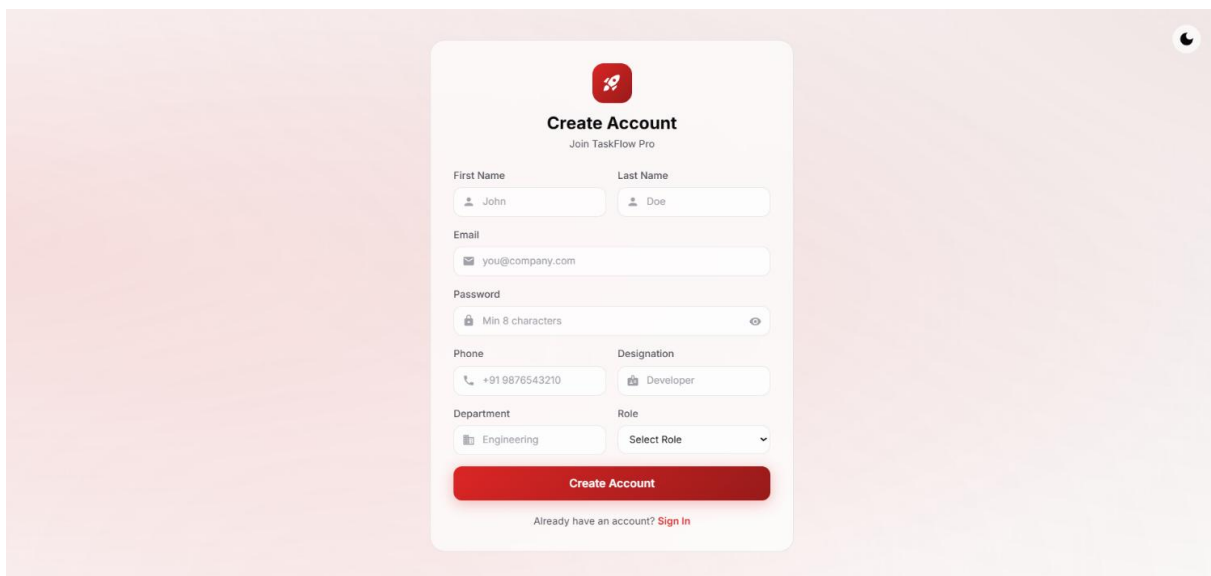
A screenshot of the TaskFlow Pro "Create Account" page. The page has the same light pink background and circle pattern as the login page. In the center is a white rounded rectangle containing the registration form. At the top is the TaskFlow Pro logo and the text "Create Account" and "Join TaskFlow Pro". The form has several fields: "First Name" (placeholder "John"), "Last Name" (placeholder "Doe"), "Email" (placeholder "you@company.com"), "Password" (placeholder "Min 8 characters" with an eye icon), "Phone" (placeholder "+91 9876543210"), "Designation" (placeholder "Developer"), "Department" (placeholder "Engineering"), and "Role" (a dropdown menu with "Select Role" selected). A red "Create Account" button is at the bottom of the form. Below the button is a link: "Already have an account? Sign In".

Figure 6.2: Registration Page

The account creation form, allowing a new user to register as either a Manager or a Team Member; the backend auto-generates a role-prefixed Employee ID (MGR/EMP) upon successful submission.

6.2 Manager Experience

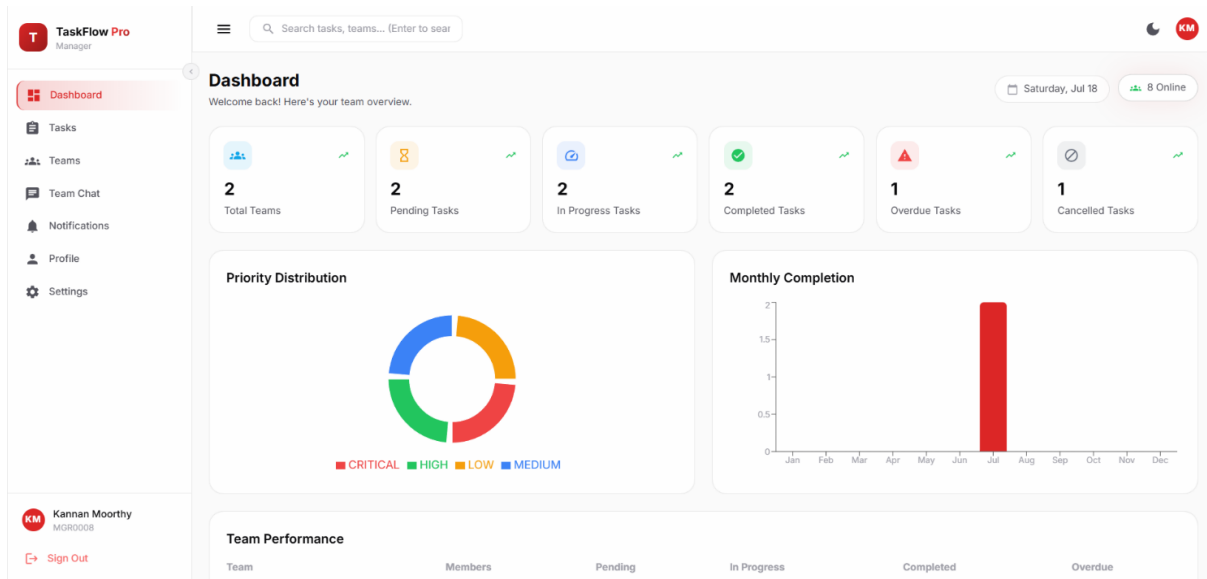


Figure 6.3: Manager Dashboard

The Manager's analytics dashboard, showing task-status count cards, a priority-distribution pie chart, a monthly completion bar chart, and a per-team performance breakdown, refreshed in real time via WebSocket.

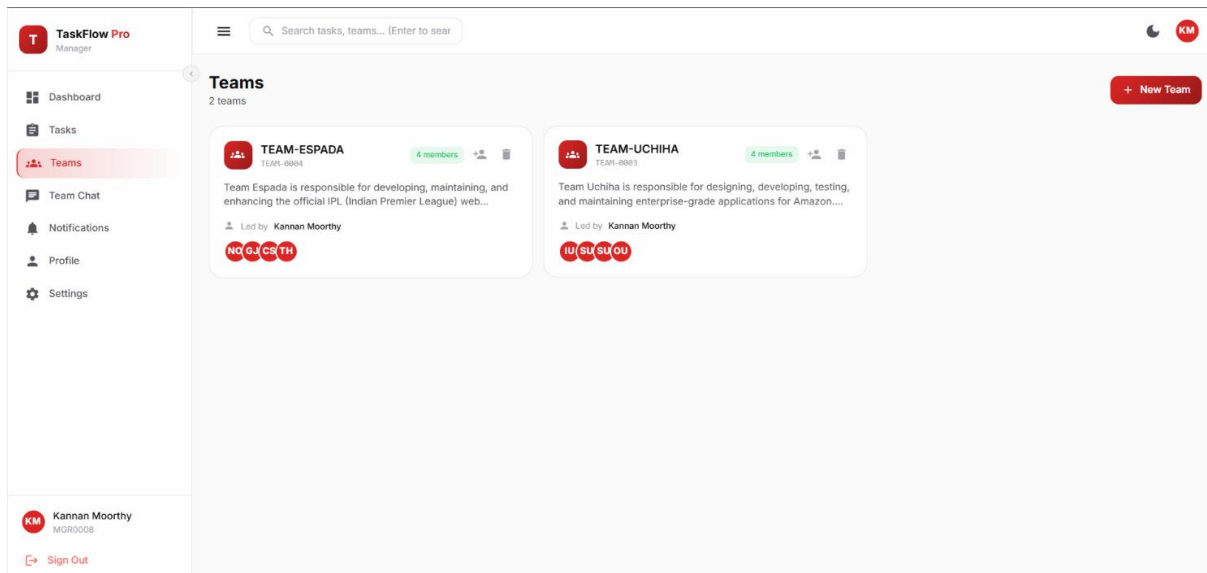


Figure 6.4: Teams Overview (Manager)

The Teams listing page for a Manager, displaying every owned team as a card with its member count, task summary, and quick-access management options.

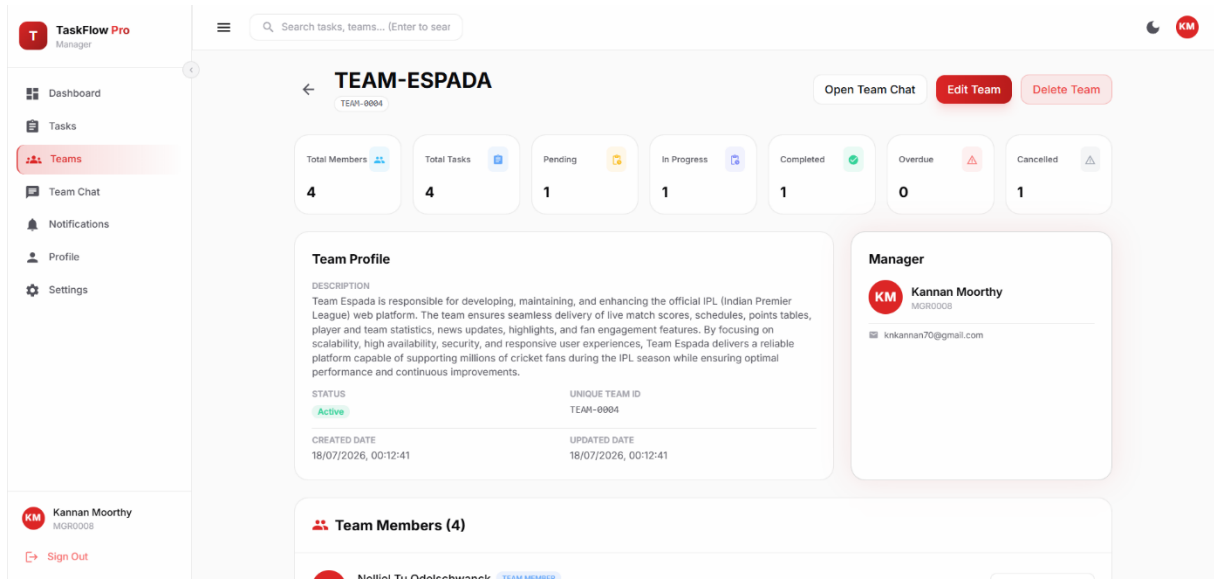


Figure 6.5: Team Detail Page

The detailed view of a single team, showing its member roster, associated tasks, and a direct link into that team's chat channel.

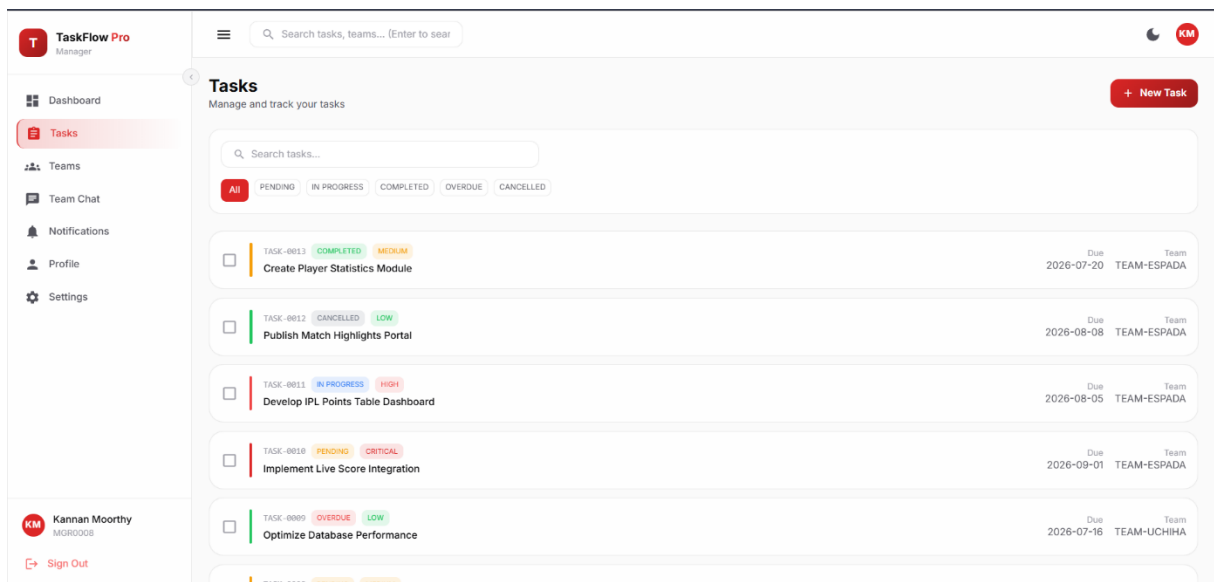


Figure 6.6: Task List (Manager)

The paginated task list with status, priority, and date-range filters, plus bulk selection controls for the bulk status update and bulk delete operations available to Managers.

TaskFlow Pro
Team Member

Search tasks, teams... (Enter to search)

TASK-0007

Build Order Management REST APIs

Description
Develop RESTful APIs for creating, updating, tracking, and managing customer orders with proper validation and exception handling.

Attachments
No attachments yet. [Add File](#)

Comments (0) History
No comments yet. Be the first to start the discussion!

Write a comment...

STATUS
In Progress
Pending
In Progress
Completed

START DATE 2026-07-19 **DUE DATE** 2026-08-04

ASSIGNED TEAM
TEAM-UCHIHA

ASSIGNED MEMBERS
Itachi Uchiha
Sasuke Uchiha
Shisui Uchiha
Obito Uchiha

Itachi Uchiha
EMP0009
Sign Out

Figure 6.7: Task Detail Page

A single task's detail view, combining its description, assignees, priority and due-date metadata, the comment thread with @mention support, file attachments, and the full audit history.

TaskFlow Pro
Manager

Search tasks, teams... (Enter to search)

New Task

Create a new task and assign it to a team

Task Title
Implement feature X...

Description
Describe the task details...

Priority
Select Priority

Assign to Team
Select Team

Start Date
dd-mm-yyyy

Due Date
dd-mm-yyyy

Estimated Hours (Optional)
e.g. 4.5

Cancel **Create Task**

Kannan Moorthy
MGR0008
Sign Out

Figure 6.8: Task Creation Form

The task creation interface used by Managers to define a new task's title, description, priority, start/due dates, and estimated hours before it is auto-assigned to the team.

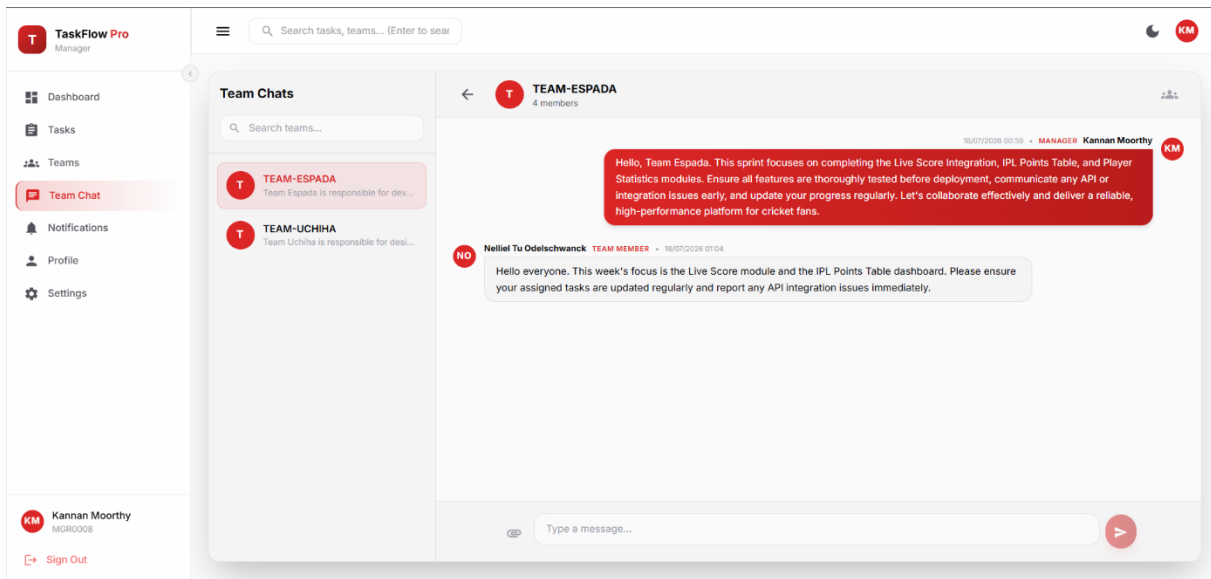


Figure 6.9: Team Chat

The real-time team chat interface, showing sender initials, name, and role badge, with the current user's messages aligned to one side and edit/delete/pin controls available inline.

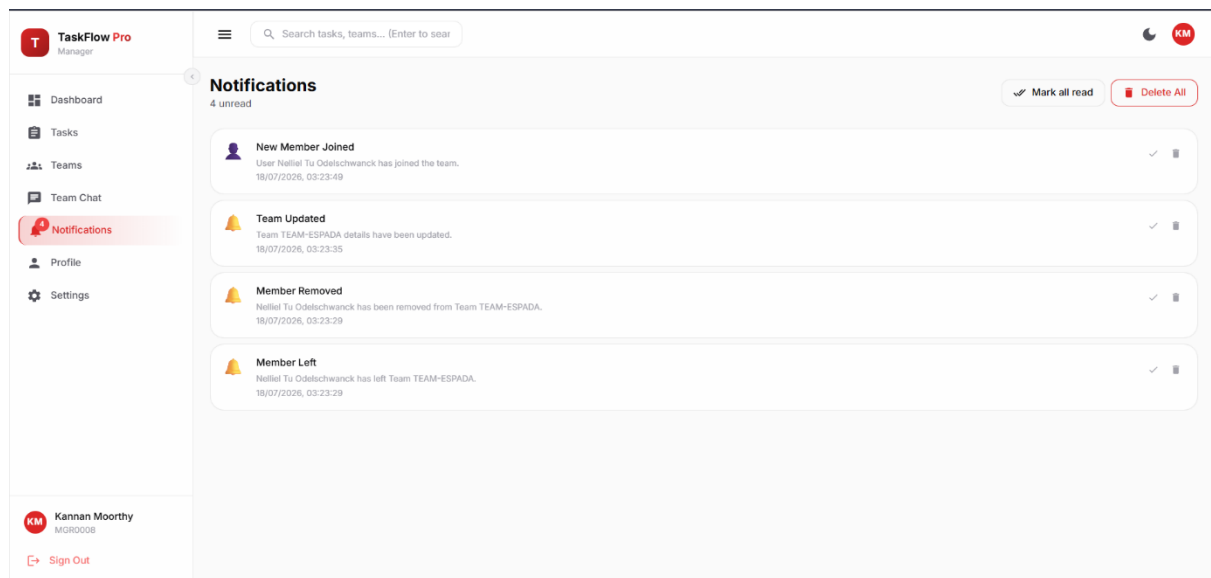


Figure 6.10: Notification Center

The in-app notification center, listing task, team, chat, and account-related events with read/unread state, an unread-count badge, and options to mark or delete notifications.

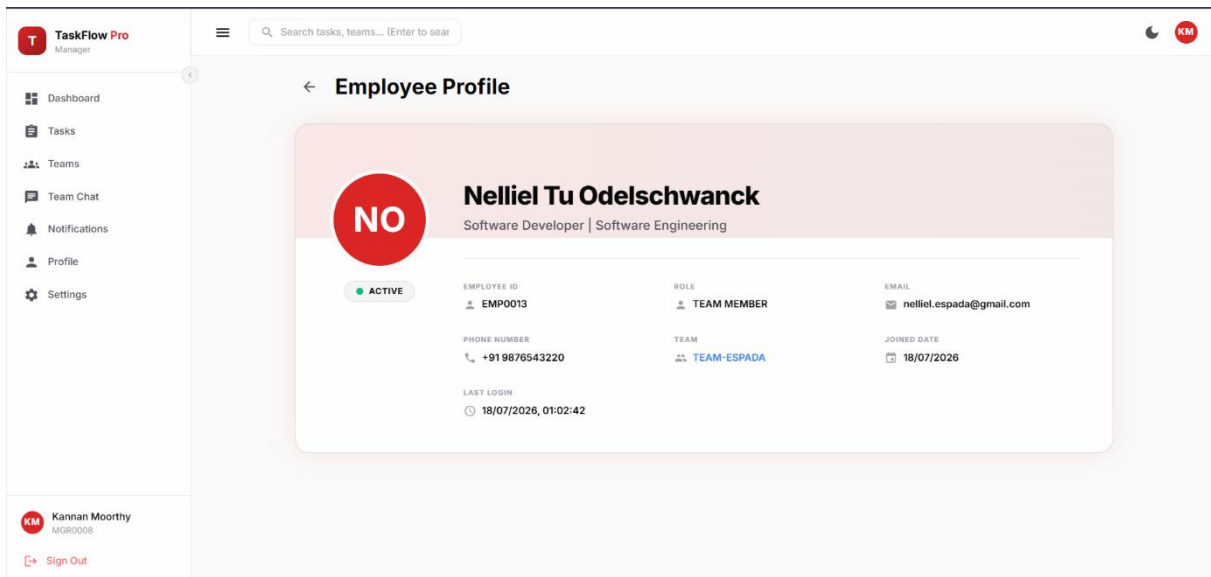


Figure 6.11: Employee Profile View

A detailed employee profile page, showing identity, contact information, designation, department, and current team assignment.

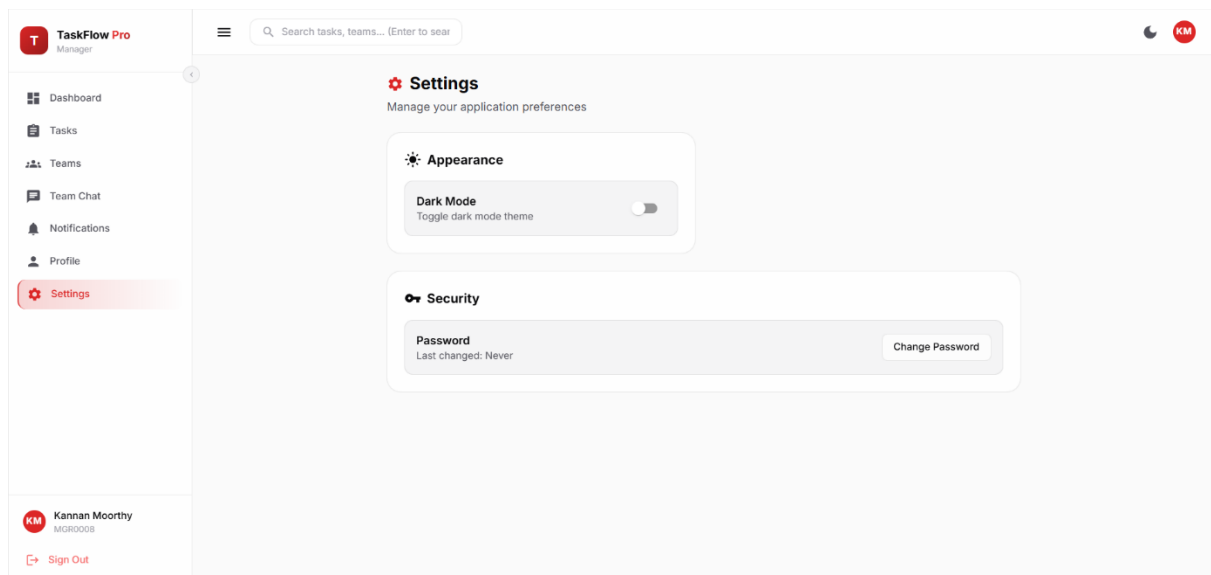


Figure 6.12: Account Settings

The account settings page, where a user changes their password after verifying their current password, backed by BCrypt hashing on the server.

6.3 Team Member Experience

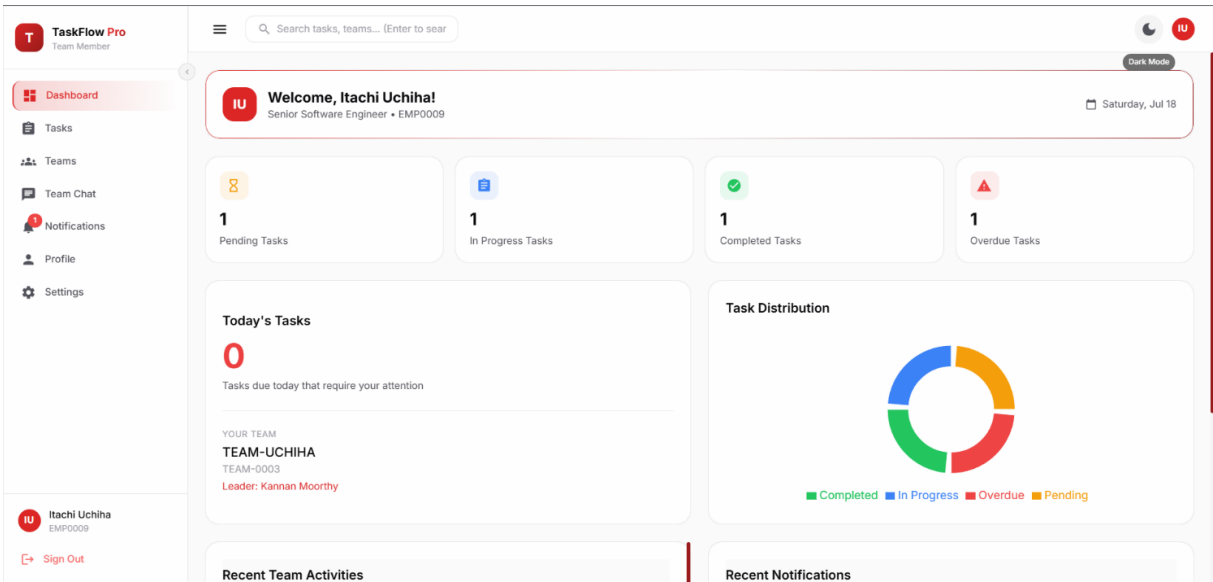


Figure 6.14: Team Member Dashboard

The personalized dashboard shown to a Team Member, summarizing their own Pending, In Progress, Completed, and Overdue task counts alongside their team information and completion progress.

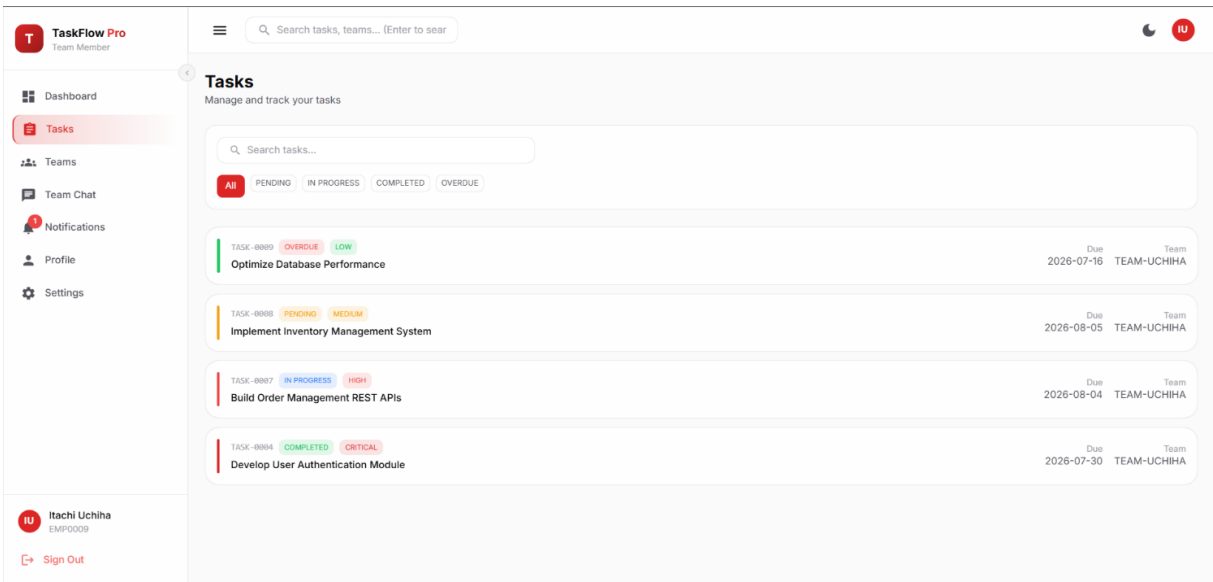


Figure 6.15: My Tasks (Team Member)

The Team Member's task list, scoped strictly to tasks assigned to them, with the same status/priority/date filtering available in a read-appropriate, self-service view.

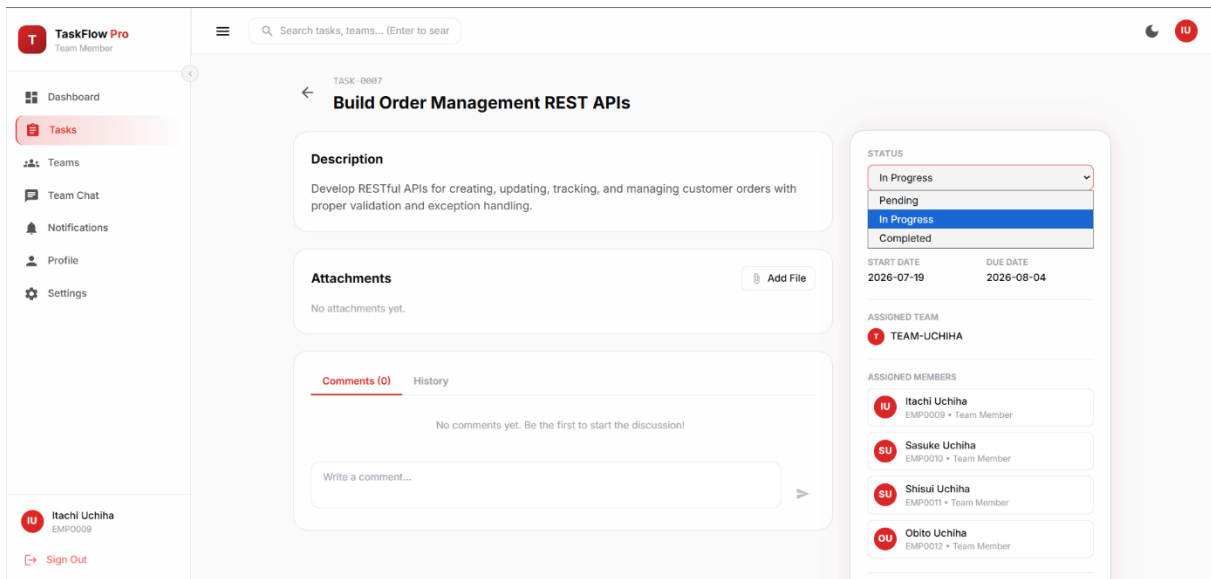


Figure 6.16: Task Detail (Team Member)

A task detail view as seen by a Team Member, permitting status and progress updates, comments, and attachment uploads, while restricting the Cancelled status to Managers only.

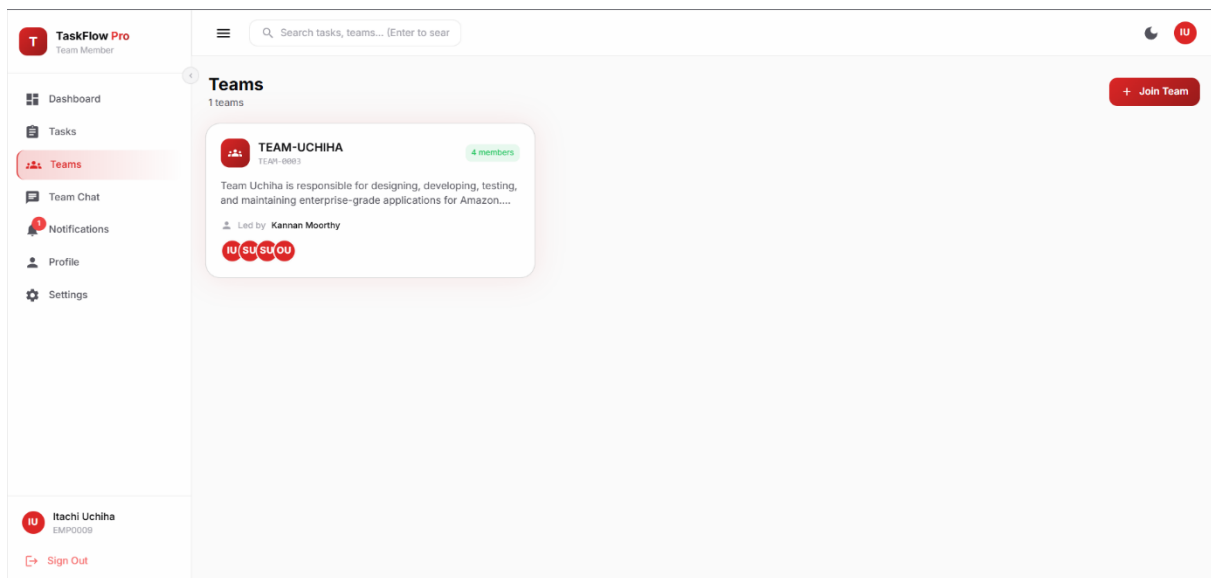


Figure 6.17: Join Team Page

The self-service Join Team page, where a Team Member enters a Team ID to join a team; the system prevents duplicate or multiple simultaneous memberships.

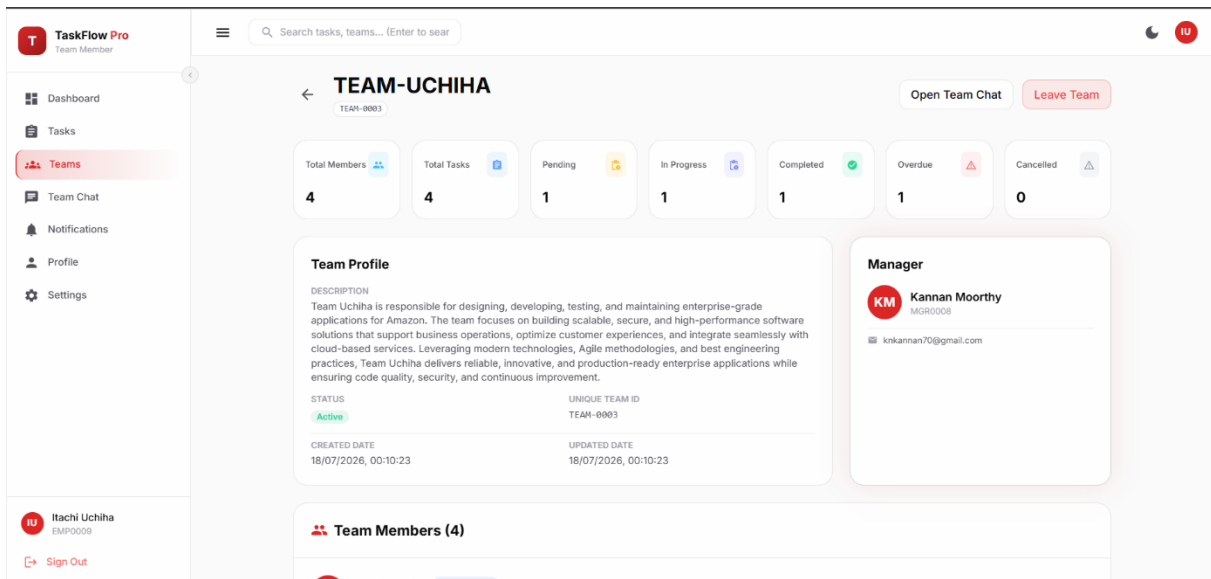


Figure 6.18: Team Detail (Team Member)

The team detail page as seen by a Team Member, scoped to their own team, showing fellow members and the team's task list.

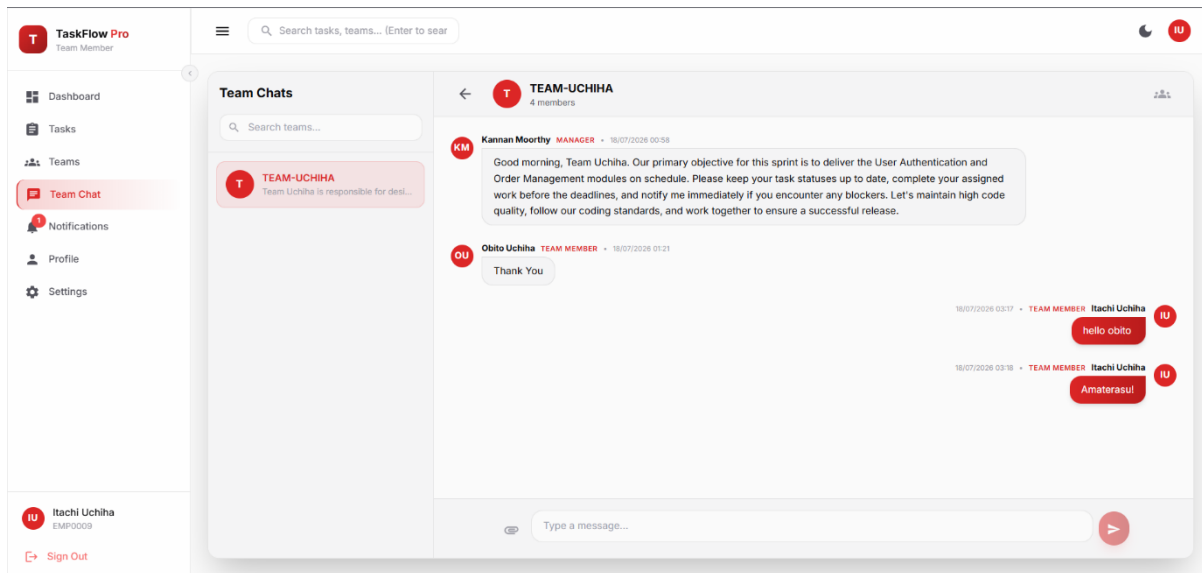


Figure 6.19: Team Chat (Team Member)

The team chat interface from a Team Member's perspective, showing the same real-time messaging, avatar, and identity display available to Managers within their team scope.

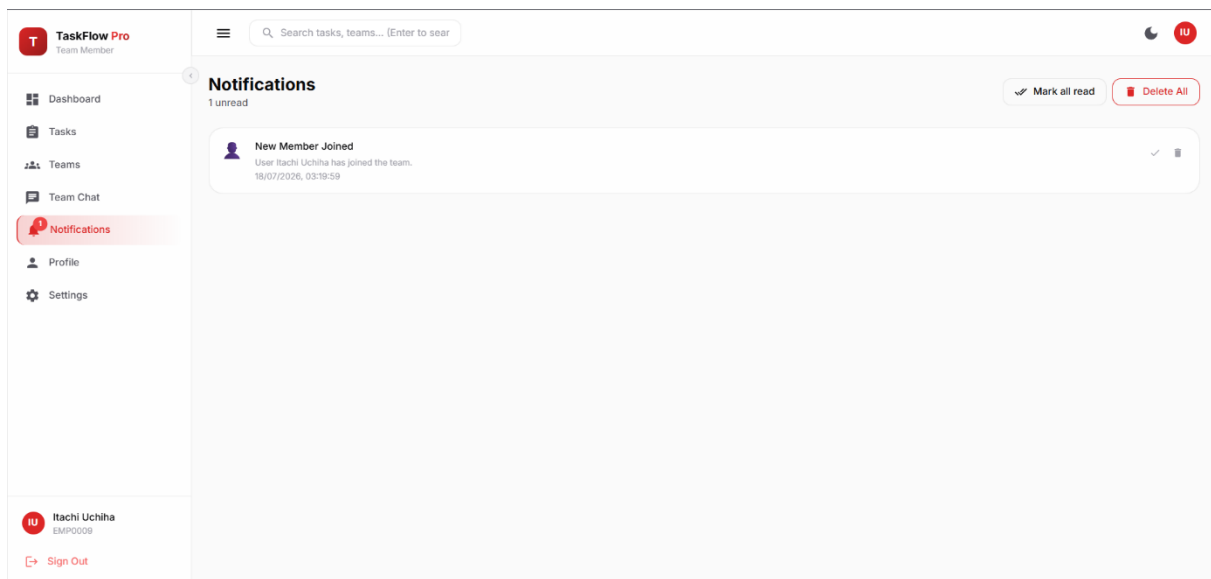


Figure 6.20: Notifications (Team Member)

The notification center as experienced by a Team Member, surfacing task assignments, mentions, and team updates relevant to their own activity.

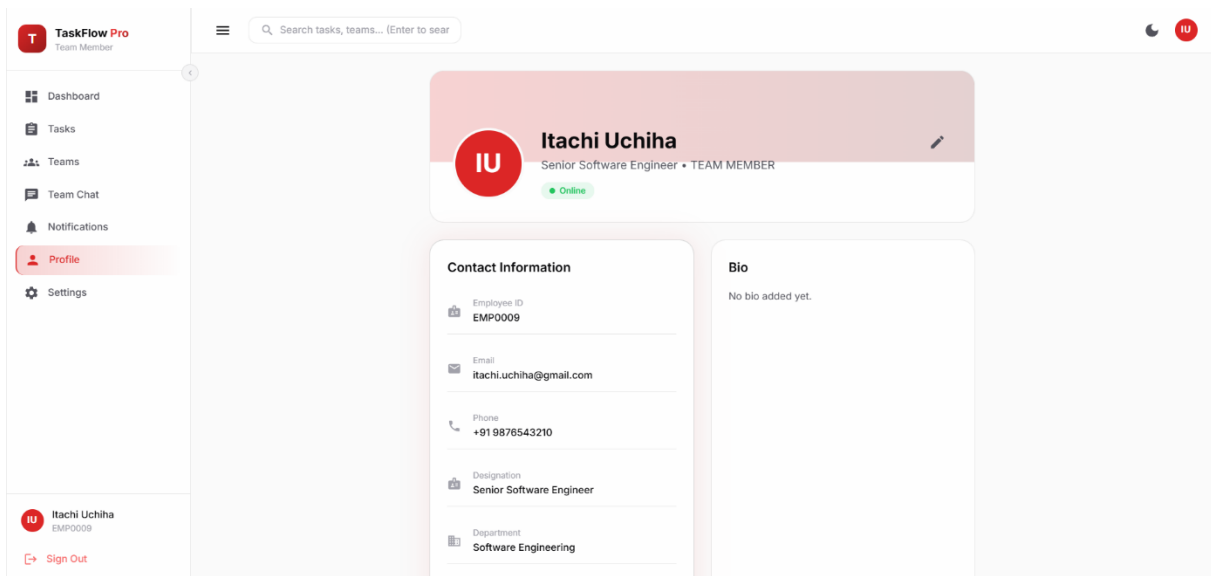


Figure 6.21: Profile Page (Team Member)

The profile management page for a Team Member, allowing them to update their designation, department, bio, skills, and avatar.

Chapter 7: Testing

7.1 Unit Testing

Unit-level testing focuses on the service layer, where the majority of the business logic and validation rules reside — for example, verifying that TeamService rejects a mutation attempt from a Manager who does not own the target team, that TaskService correctly computes the OVERDUE state for a task past its due date, and that AuthService generates unique, correctly-prefixed Employee IDs for each role.

7.2 Functional Testing

Functional testing exercises complete request/response cycles through the REST API (using the Swagger UI and integration test clients) to confirm that each endpoint behaves correctly end-to-end — including authentication, authorization rejection for out-of-role requests, and correct HTTP status codes and response payloads for both success and error paths.

7.3 Sample Test Cases

Test Case	Steps	Expected Result	Actual Result
Manager login with valid credentials	POST /auth/login with a registered Manager's email and correct password	200 OK with accessToken and refreshToken	As expected
Login with incorrect password	POST /auth/login with a valid email and incorrect password	401 Unauthorized, no token issued	As expected
Team Member attempts to create a team	POST /teams with a Team Member's access token	403 Forbidden — role check fails	As expected
Manager creates a task for owned team	POST /tasks with valid team ID owned by the Manager	201 Created; task auto-assigned to all active members	As expected
Manager mutates a team they do not own	PUT /teams/{teamId} for a team owned by a different Manager	403/400 — ownership validation fails	As expected
Concurrent task edit (optimistic lock)	Two clients fetch the same task, both submit conflicting updates	Second update rejected due to version mismatch	As expected
Password reset with expired OTP	Submit a valid OTP after the 10-minute expiry window	400 Bad Request — OTP expired	As expected
Team deletion cascade	Manager deletes an owned team with active members and chat history	Team, memberships, chat, and task assignments all removed transactionally	As expected
File attachment over size limit	Upload a file larger than 5 MB to a task	400 Bad Request — file size exceeds limit	As expected
Real-time notification delivery	Task assigned to a team while a member is connected via WebSocket	Member receives a live STOMP push without a page refresh	As expected

7.4 Test Summary

Testing across authentication, authorization, ownership validation, concurrency control, and real-time delivery confirms that the system's core guarantees — strict role separation, team-scoped data access, and transactional integrity — hold under both expected and adversarial usage patterns.

Chapter 8: Results

8.1 Application Workflow

In practice, a Manager registers and logs in, creates one or more teams (each receiving an auto-generated Team ID), and populates them either by creating new employee accounts directly or by adding existing employees via Employee ID lookup. The Manager then creates tasks against a team, which are automatically assigned to every active member and communicated by email. Team Members log in, join their assigned team, and work through their task list — updating status, adding comments, attaching files, and collaborating with teammates through the team chat. Throughout this cycle, the Manager's dashboard and the in-app notification system update in real time via WebSocket, and CSV reports can be downloaded at any point for external review.

8.2 Achievements

- A fully functional two-role RBAC system enforced consistently across both backend (@PreAuthorize) and frontend (protected routes and conditional rendering).
- End-to-end task lifecycle management, including auto-assignment, comments with mentions, file attachments, and a complete audit trail.
- A working real-time layer (WebSocket/STOMP) powering live dashboards, notifications, and team chat.
- Role-specific analytics dashboards with visual charts and one-click CSV export.
- A secure authentication system featuring short-lived JWTs, refresh-token rotation, and OTP-based password recovery.

8.3 Output

The delivered system is a working, installable full-stack application: a Spring Boot REST API exposing a documented, versioned set of endpoints, and a React single-page frontend that consumes them, together implementing every module described in Chapter 5.

8.4 Benefits

- Managers gain complete, real-time visibility into team and task performance without manual status collection.
- Team Members receive a focused, self-service workspace scoped strictly to their own responsibilities.
- Organizations gain an auditable, tamper-evident history of task-related activity.
- The modular architecture allows individual modules (e.g., Notifications, Chat) to be extended independently without destabilizing the rest of the system.

8.5 Performance

Performance-sensitive read paths, such as team chat retrieval, use @EntityGraph to eagerly fetch related sender data in a single query, avoiding N+1 query overhead. Optimistic locking on the Task entity ensures that concurrent updates are detected and rejected safely rather than being silently overwritten, preserving data integrity under concurrent access without resorting to heavier pessimistic locking strategies.

Chapter 9: Future Enhancements

The following enhancements are realistic, incremental extensions to the existing architecture, building on the modules and technologies already in place rather than requiring a redesign.

1. Full WebSocket Chat Push — extend the existing STOMP infrastructure (already used for dashboards and notifications) to remove any remaining chat polling in favor of pure push-based delivery.
2. Kanban Board View — add a drag-and-drop status-lane view over the existing Task Management module, reusing the current status enum (Pending/In Progress/Completed/Cancelled) as the lane structure.
3. Task Dependencies — introduce predecessor/successor relationships between tasks, extending the existing Task entity and history-tracking mechanism.
4. Calendar View — provide a visual, date-based scheduling view built from the existing start/due-date fields already stored on each task.
5. Advanced Analytics — extend DashboardService with burndown charts, team velocity, and longer-term productivity trend reporting.
6. Email Digest — build on the existing Spring Mail integration to add scheduled daily/weekly summary emails; the NotificationType enum already includes DAILY_SUMMARY and WEEKLY_REPORT values reserved for this purpose.
7. Mobile Application — a React Native companion app consuming the existing, already-documented REST API.
8. Role Expansion — introduce an Admin super-role and a Team Lead sub-role, extending the current two-role model without breaking existing @PreAuthorize checks.
9. In-Browser File Preview — render task attachments (images, PDFs) inline rather than requiring a download, building on the existing FileStorageService.
10. Task Templates — allow Managers to save and reuse common task configurations for recurring work.
11. Two-Factor Authentication (2FA) — add TOTP-based verification as an additional layer atop the existing JWT authentication flow.
12. Internationalization (i18n) — introduce multi-language support across the React frontend.
13. Docker Deployment — provide a one-command docker-compose setup bundling the backend, frontend, and MySQL database for simplified deployment.

Chapter 10: Conclusion

TaskFlow Pro successfully demonstrates a complete, production-oriented approach to enterprise task and team management, built on a modern, well-established technology stack — Spring Boot 3 and Java 17 on the backend, React 19 and Vite on the frontend, and MySQL as the persistence layer. The system's strict two-role RBAC model, enforced consistently at both the API and UI layers, provides clear accountability and access separation between Managers and Team Members. Its real-time WebSocket/STOMP layer keeps dashboards, notifications, and chat synchronized without manual intervention, while its comprehensive audit-history mechanism ensures that every task-related change remains traceable. Beyond meeting its stated functional requirements, the project illustrates sound software engineering practice throughout: a cleanly layered backend architecture (Controller → Service → Repository), transactional integrity for multi-step operations, optimistic concurrency control, and a self-documenting API surface via Swagger. Taken together, these qualities make TaskFlow Pro not only a functional prototype but a genuinely extensible foundation — as evidenced by the realistic, architecture-aligned enhancement roadmap identified in Chapter 9 — suitable for continued development into a fuller enterprise collaboration platform.

References

The following official resources were referenced for the technologies used in the design and implementation of this project.

1. Spring Boot Reference Documentation — <https://docs.spring.io/spring-boot/>
2. Spring Security Reference Documentation — <https://docs.spring.io/spring-security/reference/>
3. Spring Data JPA Reference Documentation — <https://docs.spring.io/spring-data/jpa/reference/>
4. Spring Framework WebSocket Support — <https://docs.spring.io/spring-framework/reference/web/websocket.html>
5. JWT (Java JWT) Library Documentation — <https://github.com/jwt/jwt>
6. Oracle Java 17 (LTS) Documentation — <https://docs.oracle.com/en/java/javase/17/>
7. MySQL 8.0 Reference Manual — <https://dev.mysql.com/doc/refman/8.0/en/>
8. Apache Maven Documentation — <https://maven.apache.org/guides/>
9. SpringDoc OpenAPI Documentation — <https://springdoc.org/>
10. React Documentation — <https://react.dev/>
11. Vite Documentation — <https://vitejs.dev/>
12. React Router Documentation — <https://reactrouter.com/>
13. TanStack Query Documentation — <https://tanstack.com/query/>
14. Material UI (MUI) Documentation — <https://mui.com/>
15. Tailwind CSS Documentation — <https://tailwindcss.com/docs>
16. Recharts Documentation — <https://recharts.org/>
17. STOMP.js Documentation — <https://stomp-js.github.io/>
18. SockJS-Client Documentation — <https://github.com/sockjs/sockjs-client>
19. Axios Documentation — <https://axios-http.com/>
20. React Hook Form Documentation — <https://react-hook-form.com/>